

うさうさ テスト&テストレポート プロンプト集

単一HTMLツール(vanilla JS・オフライン)を、セッションをまたいで 同じ品質でテスト・レポートするための、コピペ用プロンプト集です。

このプロンプト集について(WHY → WHAT → HOW)

- **WHY**(なぜ): 毎回ゼロから説明すると、テストの粒度やレポートの体裁がブレる。最小の指示で「商用テスト → 実行 → 事実/所感レポート」を再現できる土台が要る。
- **WHAT**(なにを): 用途別の8プロンプト+付録(命名規則・チェックリスト)。各プロンプトはそのまま貼って使える本文付き。
- **HOW**(どうやって): 番号順に使うのが基本。新機能を足したときは 04→05→07 を回す。レポートだけ欲しいときは 06 単独でよい。

凡例: ● かんたん(短い指示でOK) / ● ふつう(前提の共有が要る) / ● むずかしい(判断・切り分けが要る)

共通の約束(全プロンプト前提)【事実】

これを最初に一度伝えておくと、以降の指示が短くなります。

前提(うさうさ流):

- 対象は単一HTML・vanilla JS・オフライン前提。テストは jsdom + Node。
- ツール本体は window.__usausa に純粋関数を公開してある(テスト用API)。
- 出力ファイル名は ASCII のみ + 日付スタンプ(YYMMDD-HHMMSS)。
- レポートは「結論ファースト」「事実と所感の分離(【事実】/【所感】)」。
- ウソなく: 数値・項目数・合否は実行ログの実測のみ。捏造しない。
- 製品の不具合と、テストコード由来の不一致は必ず区別して書く。
- 名前は出さない(役職名のみ)。締めは高杉晋作「面白きこともなき世を面白く」。

01. テスト用APIの確認 ●

用途: 何がテストできるかを把握する。最初の一步。

このツールが `window.__usausa` で公開している関数の一覧を出して。

それぞれが「純粋関数か」「DOM依存か」「ネットワーク依存か」も分類して。

期待: 公開キー一覧 + 3分類。テスト設計の地図になる。

02. 商用テストコードを作る ●

用途: jsdom 単一ファイルの商用テストを生成。

このHTML用の商用テストを単一ファイル (`xxx.test.mjs`) で作って。

- jsdom で読み込み、`window.__usausa` を検証
- 対象HTMLは同フォルダから自動検出 (PRO版優先) + 引数/環境変数でも指定可
- カテゴリ: 単体 (純粋関数) / 防御 (サニタイズ) / 集計 / 結合 (直列化の往復) /
DOM・刻印 / モンキー (異常入力で落ちない)
- セクション見出しを出し、最後に「N passed / M failed」と失敗一覧を表示
- 全成功で終了コード0、失敗で1

作り終えたら実行して、全項目PASSを実測ログで見せて。

期待: テストファイル + 実行ログ。注意: バイナリ依存 (docx/xlsx解凍) は 手元再現が不安定なので、入れるか外すかを明示して判断させる。

03. テストを実行して全緑を確認 ●

用途: 既存テストの再実行。

テストを実行して、カテゴリ別の結果と「N passed / M failed」を実測ログで出して。

1件でも失敗したら、製品の不具合かテストコード由来かを切り分けて報告して。

期待: 実行ログ + (失敗時) 切り分け。

04. テスト結果レポートを作る 🟡

用途: 配布用のレポート (Markdown)。

さっきのテスト実行の結果を、テスト結果レポート (Markdown) にして。

- 結論ファースト (全N項目の可否を冒頭に)
- 【事実】実行概要 (日時・Node版・jsdom・対象ファイル・版・スキーマ・コマンド・終了コード)
- 【事実】カテゴリ別結果の表 (項目数・PASS・FAIL を実測値で)
- 【事実】各カテゴリで確認した内容
- テスト中に直した点があれば、製品バグかテスト側の誤りかを区別して明記
- 【所感】は事実と分けて短く
- 対象範囲と制約 (オンライン専用機能・バイナリ依存・見た目はjsdom対象外)
- 再現方法 (コマンド)
- ファイル名は ASCII + 日付 (YYMMDD-HHMMSS)、締めは高杉晋作の一言

項目数やNode版は必ず実測。記憶で埋めない。

期待: 事実/所感が分離された配布レポート。

05. 失敗の切り分け (製品 vs テスト) 🔴

用途: FAIL が出たときの誠実な原因分解。

このFAILは「製品 (ツール本体) の不具合」か「テストコードの前提誤り」か、

根拠つきで切り分けて。製品側なら最小修正案、テスト側ならテストの修正案を出して。

直したら再実行して、何件→何件になったかを実測で見せて。

判断を曖昧にせず、どちらかを言い切って(不明なら不明と言う)。

期待:原因の言い切り+修正+再実行ログ。※テスト味見のたとえ:テストは「味が薄い(製品が薄味)」のか「舌が壊れてる(テストが誤り)」のかを分けるもの 🍷 (※不正確)。

06. 新機能の回帰テストを足す 🟡

用途:機能追加のたびに最小で守りを増やす。

今足した機能(○○)の回帰テストを既存テストに追加して。

- 純粋関数があれば単体で(境界・異常値も)
- 保存に関わるなら serialize/deserialize の往復で
- UIなら該当ボタン/要素の存在+押下でエラーが出ないこと
- 防御(不正入力で落ちない)も1つ

追加後、全カテゴリを実行して全緑を実測ログで確認して。

期待:追加テスト+全緑ログ。

07. バージョンアップ+「テスト済み」刻印 🟡

用途:節目のリリース処理。

全緑を確認したら、バージョンを1つ上げて刻印して。

- 刻印箇所をすべて更新(タイトル・先頭コメント・表紙・クレジット・
バグ報告・起動ログ・__ver定数など)
- 刻印に「テスト済み」を入れる

- ファイル名は PRO+当日日付時刻(YYMMDD-HHMMSS)で保存

- テストの版数チェックも合わせて更新し、もう一度全緑を実測で確認

期待: 刻印更新+改名ファイル+再テスト緑。

08. レポートの読者別バリエーション ●

用途: 相手に合わせて出し分け。

このテスト結果レポートを、次の読者向けに作り分けて:

(A) 上長/人事担当むけ: 1ページ、結論と品質状況だけ、専門用語控えめ

(B) 講師/開発者むけ: カテゴリ別・再現コマンド・既知の制約まで詳細

どちらも事実と所感を分け、数値は実測のみ。

期待: 読者別の2版。役職名のみで、個人名は出さない。

付録A. 命名規則【事実】

種類	例
ツール本体(PRO)	usausa-prompter-PRO-YYMMDD-HHMMSS.html
テストコード	usausa-prompter.test.mjs
実行手順	TESTING.md
テスト結果レポート	usausa-prompter-test-report-YYMMD-HHMMSS.md

- ファイル名は ASCII のみ。日本語・空白・スラッシュを入れない。
- 日付スタンプは当日の実時刻(YYMMDD-HHMMSS)。

付録B. 仕上げ前チェックリスト【事実】

- ☐ 全カテゴリを実行し、N passed / 0 failed を実測で確認した
- ☐ FAIL は「製品」か「テスト」かを切り分け、レポートに区別して書いた
- ☐ 項目数・Node版・日時は実測（記憶で埋めていない）
- ☐ 刻印箇所をすべて更新し「テスト済み」を入れた
- ☐ レポートは結論ファースト＋事実/所感の分離
- ☐ 出力ファイル名は ASCII＋日付
- ☐ オンライン専用機能・バイナリ依存・見た目の制約を明記した
- ☐ 個人名なし（役職名のみ）／締めに高杉晋作の一言

付録C. 落とし穴メモ【所感】

- jsdom は ESM で `NODE_PATH` を見ない。手元では `npm i jsdom`（ローカル）が確実。
- jsdom はタイマー等で終了しないことがある。テスト末尾で `process.exit` する。
- 見た目（配色・アニメ・画像）はjsdomでは判定できない。実機確認を前提に。
- 「全部入れる」より「手元で安定して回る」を優先。再現できないテストは外して明記。

面白きこともなき世を面白く 🐰